

Лабораторная работа № 8 по курсу дискретного анализа: Жадные алгоритмы

Выполнил студент группы М8О-308Б-22 МАИ *Немкова Анастасия*.

Условие

На первой строке заданы два числа, N и $p > 1$, определяющие набор монет некоторой страны с номиналами $p^0, p^1, \dots, p^{N-1}$. Нужно определить наименьшее количество монет, которое можно использовать для того, чтобы разменять заданную на второй строчке сумму денег $M \leq 2^{32} - 1$ и распечатать для каждого i -го номинала на i -ой строчке количество участвующих в размене монет.

Метод решения

Жадный алгоритм решает задачу, принимая локально оптимальные решения на каждом этапе, с надеждой, что они приведут к глобально оптимальному решению.

В данной задаче, начиная с самой большой степени и двигаясь к меньшим, мы вычитаем из заданной суммы текущий номинал, пока это возможно, увеличивая счетчик для каждого номинала. Жадный алгоритм подходит для этого случая, так как номиналы образуют возрастающую последовательность степеней p . Для чисел, образующих геометрическую прогрессию, жадный выбор оптимален, так как каждый номинал можно представить как кратное более мелкого номинала. Поэтому, начиная с наибольшего номинала, мы всегда получаем минимальное количество вычитаний.

Если номиналы заданы произвольно и не имеют структуры геометрической прогрессии, то жадный подход может не привести к оптимальному решению. Например, для номиналов 1, 3, 4 значения $m = 6$, жадный алгоритм сначала выберет номинал 4 (1 раз), а затем дважды 1 (всего 3 слагаемых). Однако оптимальное решение — использовать два номинала 3, что даст 2 слагаемых.

Чтобы решить задачу для произвольного набора номиналов, можно использовать динамическое программирование. Суть алгоритма заключается в том, что мы в массиве будем хранить минимальное количество номиналов, необходимых для получения суммы для каждого значения от 1 до m . В результате, массив будет содержать минимальное количество номиналов, необходимых для получения суммы m .

Описание программы

Программа считывает три целых числа: n (количество номиналов), p (основание для степеней) и m (число, которое нужно разложить).

Создается вектор `nominals`, в который записываются степени числа p от p^0 до p^{n-1} . Это делается с помощью цикла, который последовательно умножает предыдущее значение на p .

Далее инициализируется вектор `counts`, который будет хранить количество использованных номиналов для каждого значения. Затем, начиная с самого большого номинала, пытаемся вычесть его из `m`, пока `m` неменьше текущего номинала, увеличивая счетчик в `counts` для каждого использованного номинала.

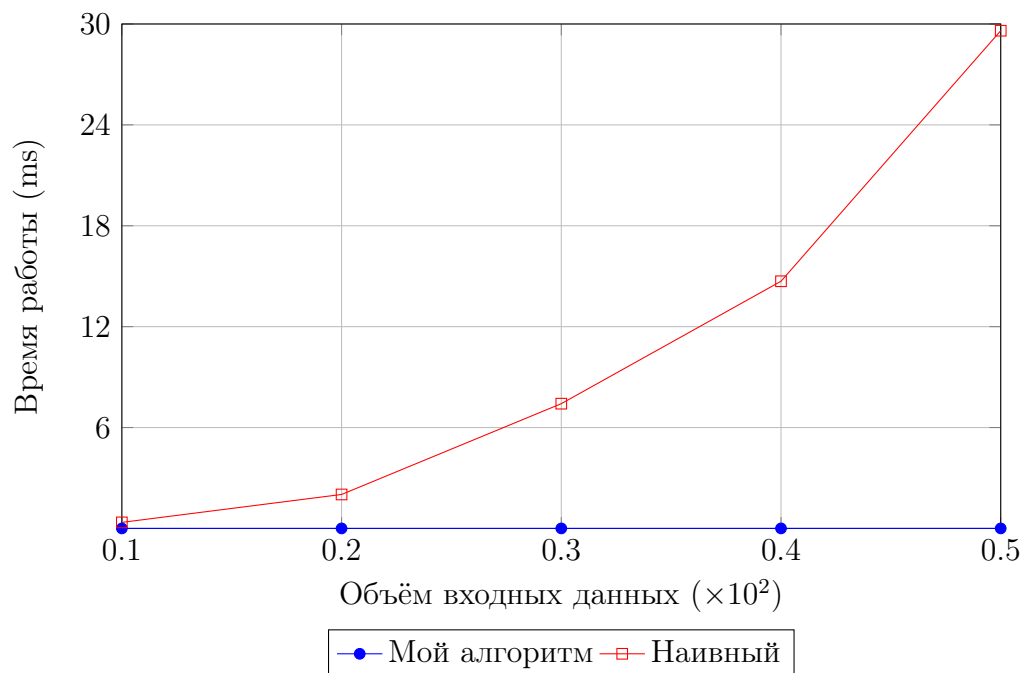
Сложность: $O(nk)$, где n — количество степеней, а k — максимальное количество раз, которое может понадобиться для уменьшения `m` на номинал. В худшем случае, k может быть пропорционально `m` (если все номиналы маленькие).

Дневник отладки

Были произведены попытки улучшить алгоритм, а также использовать динамическое программирование.

Тест производительности

Для измерения производительности сравним время работы данного алгоритма и наивного, в котором перебираем все возможные количества монет для каждого номинала и вычисляем сумму, создаваемую каждой комбинацией. Протестируем на входных данных, где $p = 3$, $N = k * 10$ и $M = k * 10^2$ ($k = 1, \dots, 5$)



Из графика видим, что жадный алгоритм имеет линейную сложность ($O(n)$), а наивный экспоненциальную ($O(n * m^n)$)

Выводы

В ходе данной лабораторной работы были разобраны жадные алгоритмы и их применение на примере размена денег с использованием монет различных номиналов.

Основное отличие жадных алгоритмов от динамического программирования заключается в том, что жадные алгоритмы принимают решения на каждом шаге, основываясь на локально оптимальных выборах, в то время как динамическое программирование рассматривает более глобальный подход, учитывающий все возможные состояния для достижения оптимального решения. Это позволяет жадным алгоритмам часто работать быстрее и использовать меньше памяти, но они могут не находить оптимальное решение в задачах, где это требуется.